

Documentación Técnica Backend – App de Alimentación Saludable

1. Introducción

Este documento describe la arquitectura y funcionamiento del backend del proyecto de aplicación móvil enfocada en ayudar a los usuarios a mejorar su alimentación y llevar una vida más saludable.

La aplicación utiliza datos personales del usuario como edad, peso, objetivos nutricionales y enfermedades metabólicas para generar recomendaciones de comidas personalizadas.

El backend está desarrollado utilizando Ruby on Rails en modo API. El frontend se desarrolló con React y la base de datos utiliza PostgreSQL. Además, el sistema integrará un modelo de inteligencia artificial para generar recomendaciones nutricionales más personalizadas.

2. Objetivo del Backend

El backend es el encargado de gestionar toda la lógica del sistema y actuar como intermediario entre el frontend y la base de datos.

Sus principales funciones son:

- Registro e inicio de sesión de usuarios
- Almacenamiento de datos personales y de salud
- Gestión de comidas y valores nutricionales
- Comunicación con el sistema de inteligencia artificial
- Generación de recomendaciones personalizadas
- Exposición de endpoints REST para el frontend

3. Tecnologías Utilizadas

Ruby on Rails (API Mode) Framework utilizado para desarrollar el servidor y la lógica del backend.

React Framework utilizado para el desarrollo del frontend de la aplicación.

PostgreSQL Base de datos relacional utilizada para almacenar la información del sistema.

JSON Formato utilizado para el intercambio de datos entre frontend y backend.

Claude AI Modelo de inteligencia artificial que se utilizará para generar recomendaciones nutricionales personalizadas basadas en el perfil del usuario.

4. Arquitectura del Sistema

Usuario
Aplicación React (Frontend)
API Ruby on Rails (Backend)
Servicio de IA (Claude)
PostgreSQL (Base de Datos)

5. Arquitectura MVC en Ruby on Rails

- Ruby on Rails utiliza el patrón de arquitectura MVC (Model - View - Controller).
- Model Representa las entidades de la base de datos y contiene la lógica de negocio.
- Controller Recibe las solicitudes HTTP del frontend, ejecuta la lógica necesaria y devuelve respuestas.
- Routes Define los endpoints de la API.
- Database Contiene la estructura de datos del sistema y las migraciones.

6. Estructura del Backend

```
app/  
  controllers/  
  models/  
  services/  
  serializers/  
  
config/  
  routes.rb  
  
db/  
  migrate/  
  schema.rb
```

7. Modelo de Base de Datos

- Las principales tablas del sistema incluyen:
- users Contiene información del usuario como edad, peso, altura y objetivos nutricionales.
 - meals Contiene las comidas disponibles y su información nutricional.
 - metabolic_conditions Almacena enfermedades metabólicas del usuario como diabetes o hipertensión.

recommendations Guarda las recomendaciones de comidas generadas por el sistema.

8. Endpoints de la API

```
POST /users
Registrar usuario

POST /login
Autenticar usuario

GET /meals
Obtener comidas disponibles

GET /recommendations
Obtener recomendaciones personalizadas
```

9. Ejemplo de Request y Response

```
REQUEST

POST /users

{
  "name": "Juan",
  "age": 25,
  "weight": 70,
  "goal": "perder peso"
}

RESPONSE

{
  "status": "success",
  "message": "Usuario creado correctamente"
}
```

10. Integración de Inteligencia Artificial (Claude)

El sistema integrará un modelo de inteligencia artificial llamado Claude para generar recomendaciones de dietas personalizadas.

La IA se utilizará para analizar los datos del usuario y generar sugerencias de comidas saludables basadas en:

• Objetivo nutricional del usuario • Peso, edad y características físicas • Enfermedades metabólicas • Restricciones alimenticias

Flujo de funcionamiento de la IA:

1. El usuario registra su información en la aplicación. 2. El frontend envía los datos al backend mediante la API. 3. El backend procesa los datos y construye un prompt estructurado. 4. El backend

envía esta información al servicio de Claude mediante su API. 5. Claude analiza los datos y genera recomendaciones de dieta. 6. El backend recibe la respuesta de la IA. 7. Las recomendaciones se guardan en la base de datos. 8. El backend envía las recomendaciones al frontend para mostrarlas al usuario.

Esta integración permite que el sistema genere planes alimenticios más personalizados y adaptados a cada usuario.

11. Seguridad

Para proteger la información del usuario se implementan varias medidas de seguridad:

- Validación de datos en el backend
- Control de acceso a endpoints protegidos
- Manejo seguro de datos personales
- Autenticación de usuarios

12. Conclusión

El backend desarrollado con Ruby on Rails permite gestionar de manera eficiente los datos del sistema y proporcionar servicios al frontend mediante una API REST.

La integración con inteligencia artificial permitirá generar recomendaciones nutricionales más avanzadas, lo cual mejora significativamente la experiencia del usuario y el valor del sistema.